

Day 36 — Line-by-Line Syntax Explanation

day36_neural_network_fundamentals.py, segment by segment, line by line. Pure Python/NumPy/matplotlib syntax focus — what each construct does mechanically. Neural-network theory is covered separately in the main Day 36 study guide PDF; this document only answers "what is this line of code actually doing?"

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 1 reference

This document walks through the script top to bottom in the same order the code actually runs. Each segment below corresponds to one numbered section of the script (matching its own comment banners). Inside each segment, code is broken into small chunks — usually one to a handful of related lines. Wherever a chunk implements a real mathematical formula (the perceptron rule, an activation function, backpropagation's gradients, cross-entropy loss), a purple **MATH** box appears first showing that formula, immediately followed by the exact code that implements it, then a plain-English syntax explanation — so you can see the math converted into code, one piece at a time, in reading order.

MATH box legend: $\times$ = matrix multiply / times W_1, b_1 = weights/bias of layer 1, etc. X^T = transpose of X
 $\text{sum}[...]$ = add up over all examples or classes m = number of training examples

Segment 0 — Imports and Global Setup (lines 32–38)

```
import numpy as np
```

Imports the NumPy library under the alias **np**. Every array operation in this script — dot products, matrix multiplication, elementwise math — goes through this alias.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

The order matters: **matplotlib.use("Agg")** must run before **import matplotlib.pyplot**. It switches matplotlib to a non-interactive backend that renders straight to a file instead of opening a GUI window — this is exactly why running the script never pops up a chart on screen; it silently writes PNGs to disk instead. **plt** is the conventional alias for the plotting submodule.

```
from sklearn.datasets import make_moons
```

Imports one specific function directly out of **sklearn.datasets**, rather than importing the whole module. **make_moons** generates a synthetic 2D dataset shaped like two interleaving crescents — used later as a harder, non-linear test problem.

```
np.random.seed(42)
```

Fixes NumPy's global random-number generator to a starting point. Any later call to **np.random.*** (weight initialization, for example) produces the exact same sequence of "random" numbers on every run — which is why your terminal output matches the reference output exactly, every single time you run the file.

Segment 1 — The Perceptron Class (lines 45–73)

```
class Perceptron:
    """A single linear unit..."""
```

class Perceptron: begins a new class definition — a blueprint for objects that bundle data (weights, bias) with behavior (forward pass, training). The triple-quoted string directly under the class line is a **docstring**; Python stores it as **Perceptron.__doc__**, viewable via **help(Perceptron)**.

```
def __init__(self, n_inputs, lr=0.1):
    self.w = np.zeros(n_inputs)
    self.b = 0.0
    self.lr = lr
```

__init__ is the constructor — it runs automatically the moment you write **Perceptron(2)**. **self** is the instance being built and must be the first parameter of every method. **lr=0.1** is a **default argument**, so callers may omit it. **np.zeros(n_inputs)** builds a 1D array of that length filled with **0.0** — e.g. **np.zeros(2)** → **[0.0, 0.0]** — which becomes the weight vector. Writing **self.w = ...** attaches that array to the instance as an attribute so it persists across method calls, instead of disappearing once **__init__** returns.

MATH

$$z = (w_1 \times x_1) + (w_2 \times x_2) + b \quad (\text{general form: } z = \sum_i (w_i \times x_i) + b)$$
$$y_{\text{pred}} = 1 \text{ if } z \geq 0, \text{ else } 0$$

```
def forward(self, x):
    z = np.dot(x, self.w) + self.b
    return 1 if z >= 0 else 0
```

np.dot(x, self.w) computes the dot product: multiply matching elements together, then sum the results — **x[0]*w[0] + x[1]*w[1]**. Adding **self.b** gives the raw weighted sum **z**. The return line is a **conditional expression** (a one-line if/else that evaluates to a value): if **z >= 0** the expression evaluates to **1**, otherwise **0**. This is the perceptron's step-function activation.

```
def train(self, X, y, epochs=20):
    history = []
    for _ in range(epochs):
        errors = 0
        for xi, yi in zip(X, y):
```

history = [] starts an empty list that will collect one number per training epoch. **for _ in range(epochs):** loops exactly **epochs** times; the underscore **_** is a Python convention meaning "this loop variable's value is never used." **for xi, yi in zip(X, y):** — **zip** pairs up two sequences position by position (first element of **X** with first of **y**, second with second, and so on), letting you loop over both arrays in lockstep. **xi** is one training example (a length-2 array like **[0, 1]**), **yi** is its matching label.

MATH

The perceptron learning rule (only applied when the prediction is wrong):

$$w = w + lr \times (y - y_pred) \times x$$

$$b = b + lr \times (y - y_pred)$$

```
pred = self.forward(xi)
error = yi - pred
if error != 0:
    self.w += self.lr * error * xi
    self.b += self.lr * error
    errors += 1
```

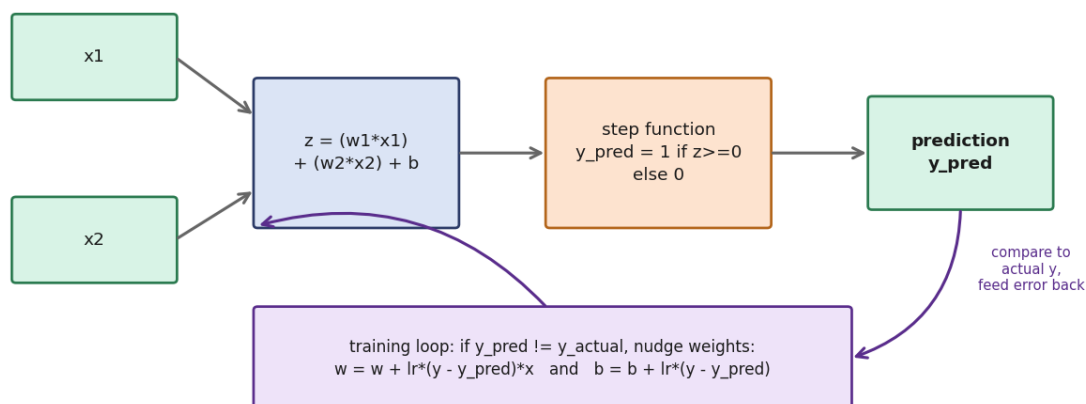
self.w += self.lr * error * xi is the perceptron learning rule itself, written as one line of NumPy. **+=** is augmented assignment, shorthand for **self.w = self.w + (...)**. Because **xi** is a NumPy array, the expression **self.lr * error * xi** (scalar × scalar × array) produces an array, so this single line updates every weight at once via NumPy's broadcasting — no explicit loop over individual weights is written. **errors += 1** is a plain integer counter increment, tallying how many examples this epoch got wrong.

```
history.append(errors)
if errors == 0:
    break
return history
```

history.append(errors) adds this epoch's error count onto the end of the list. **break** immediately exits the nearest enclosing **for** loop — once an entire pass through the data makes zero mistakes, training is finished and there's no reason to run the remaining epochs. **return history** sends the full list of per-epoch error counts back to whoever called **.train(...)**.

SYSTEM DIAGRAM

Perceptron: system diagram (Segment 1)



This is Segment 1's math and code turned into one picture: **x1/x2** flow into the **z = w·x + b** box (that's **np.dot(x, self.w) + self.b**), the step-function box thresholds it into **y_pred** (that's the **1 if z >= 0 else 0** line), and the purple loop at the bottom is **train()**'s weight update — the error feeds back and nudges the first box on the next pass through the data.

Segment 2 — demo_perceptron_and_gate() (lines 76–91)

```
X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])
```

np.array([[...], [...], ...]) — passing a list of lists turns it into a 2D array: 4 rows, 2 columns here, where each inner list is one training example's two inputs. **y_and** is a plain 1D array holding the matching AND-gate outputs for each of those 4 rows.

```
p = Perceptron(n_inputs=2)
history = p.train(X_and, y_and)
```

Perceptron(n_inputs=2) calls `__init__` using a **keyword argument** — explicit about which parameter the value **2** fills, clearer than the positional form **Perceptron(2)**. **p.train(...)** calls the instance method on **p**, mutating its internal **w** and **b** in place and returning the error history list.

```
print(f"Converged in {len(history)} epochs (errors per epoch: {history})")
```

An **f-string (f"..."**): anything inside **{}** is evaluated as a live Python expression and substituted into the string. **len(history)** counts how many epochs actually ran before the early **break** in **train()** fired.

```
for xi, yi in zip(X_and, y_and):
    print(f" {xi} -> predicted {p.forward(xi)}, actual {yi}")
```

Same **zip** pattern as inside **train()**, reused here purely to print each input/prediction/actual-label triple after training has already finished (this loop does not train anything further).

Segment 3 — demo_perceptron_xor_failure() (lines 98–114)

Structurally this mirrors Segment 2 (same **Perceptron** class, different data: **y_xor = [0, 1, 1, 0]**). Two new syntax constructs appear here that didn't show up in the AND-gate demo:

```
correct = sum(p.forward(xi) == yi for xi, yi in zip(X_xor, y_xor))
```

The parenthesized part — **(p.forward(xi) == yi for xi, yi in zip(...))** — is a **generator expression**: like a list comprehension but with **()** instead of **[]**, so it produces values one at a time instead of building a whole list in memory first. **p.forward(xi) == yi** evaluates to **True** or **False** for each pair. In Python, **True** behaves as **1** and **False** as **0** when summed, so **sum(...)** here counts how many of the 4 predictions were correct.

```
print(f"Ran {len(history)} epochs, never converged (errors per epoch: {history[:10]}...)")
```

history[:10] is a **slice** — it takes the first 10 elements of the list (or fewer, if the list is shorter than 10). This keeps the printed line short instead of dumping all 50 epochs' worth of error counts to the terminal.

Segment 4 — Activation Functions (lines 121–159)

MATH

$\text{sigmoid}(z) = 1 / (1 + \exp(-z))$ - squashes any real number into the open interval (0, 1)

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

A plain **module-level function** — no class, no **self** — called directly as **sigmoid(some_array)**. **np.exp(-z)** computes $e^{(-z)}$ **elementwise** when **z** is an array; no Python loop is needed because NumPy vectorizes the operation internally. The full expression **1 / (1 + np.exp(-z))** also broadcasts elementwise, producing an output array the same shape as the input.

MATH

$\text{ReLU}(z) = \max(0, z)$

```
def relu(z):  
    return np.maximum(0, z)
```

np.maximum(0, z) compares **0** against every element of **z** and keeps whichever is larger, elementwise — negative values become 0, positive values pass through unchanged. This is different from Python's built-in **max()**, which only compares single scalar values, not whole arrays.

MATH

$\text{softmax}(z)_i = \exp(z_i - \max(z)) / \text{sum over all } j \text{ of } \exp(z_j - \max(z))$
(the " $-\max(z)$ " shift is the numerical-stability trick; it does not change the result)

```
def softmax(z):  
    ez = np.exp(z - np.max(z)) # subtract max for numerical stability  
    return ez / ez.sum()
```

np.max(z) finds the single largest value across the whole array. **z - np.max(z)** subtracts that one scalar from every element (broadcasting again), shifting values so the largest becomes 0 — this prevents **np.exp()** from overflowing on large inputs, without changing the final softmax result mathematically. **ez.sum()** adds every element together; dividing by it normalizes the array so all entries sum to exactly 1 — a valid probability distribution.

```
for name, fn in [("sigmoid", sigmoid), ("tanh", tanh), ("relu", relu)]:  
    vals = [round(float(fn(np.array([zv]))[0]), 4) for zv in [-2, 0, 2]]  
    print(f" {name:8s}: {vals}")
```

[("sigmoid", sigmoid), ...] is a list of tuples pairing a display name (a string) with the actual function object itself — in Python, functions are first-class values, so writing **sigmoid** with no parentheses refers to the function, not a call to it. **for name, fn in [...]** unpacks each tuple into two loop variables per iteration. The middle line is a **list comprehension**: it builds a new list by evaluating the left-hand expression once for each item in

[-2, 0, 2]. np.array([zv]) wraps a single number into a 1-element array (matching the array-shaped input every activation function expects), **fn(...)** calls the function on it, and **[0]** pulls the one resulting number back out. **float(...)** converts a NumPy scalar to a plain Python float; **round(..., 4)** rounds to 4 decimal places. Finally, **{name:8s}** is an f-string **format spec**: left-pad the string to 8 characters wide so the printed columns line up.

```
z_vals = np.linspace(-5, 5, 200)
```

np.linspace(start, stop, num) generates **num** evenly spaced numbers between **start** and **stop**, inclusive — here 200 points from -5 to 5, used as the x-axis for the activation-function plots.

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.2))
```

plt.subplots(1, 3, ...) creates one figure containing a 1-row-by-3-column grid of subplots in a single call, returning **fig** (the overall figure object) and **axes** (an array of the 3 individual subplot objects). **figsize=(12, 3.2)** sets the overall image dimensions in inches.

```
pairs = [("Sigmoid", sigmoid), ("Tanh", tanh), ("ReLU", relu)]
for ax, (name, fn) in zip(axes, pairs):
    ax.plot(z_vals, fn(z_vals))
    ax.set_title(name)
    ax.axhline(0, color="gray", linewidth=0.5)
    ax.axvline(0, color="gray", linewidth=0.5)
```

zip(axes, [...]) pairs each of the 3 subplot axes with a (name, function) tuple; **(name, fn)** inside the loop header unpacks that tuple in the same step as unpacking **ax**. **ax.plot(z_vals, fn(z_vals))** draws the curve onto that specific subplot — **fn(z_vals)** calls the activation function on the whole 200-element array at once, producing all 200 y-values in one vectorized call. **ax.axhline(0, ...)** and **ax.axvline(0, ...)** each draw a thin reference line through 0, horizontal and vertical respectively.

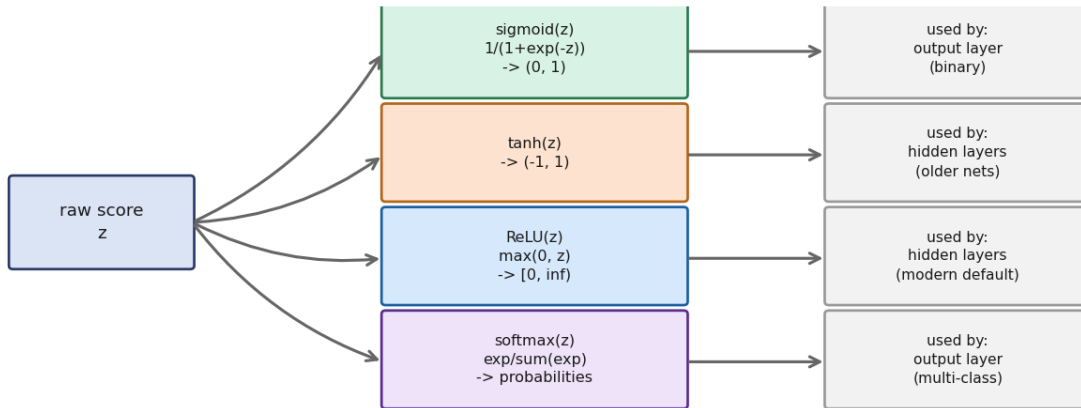
```
plt.tight_layout()
plt.savefig("activation_functions.png", dpi=110)
plt.close()
```

plt.tight_layout() automatically adjusts subplot spacing so titles and labels don't overlap each other.

plt.savefig(path, dpi=110) writes the current figure to disk as a PNG at 110 dots-per-inch. **plt.close()** releases the figure from memory once you're done with it — good practice in any script that generates multiple plots in sequence.

SYSTEM DIAGRAM

Activation functions: flow diagram (Segment 4)



The four **math** formulas from this segment, side by side as a system: a raw score z branches into whichever activation function is chosen, each one squashing it into a different output range — and each range is exactly why that function gets used where it does (sigmoid/softmax for outputs, ReLU as the modern default for hidden layers).

Segment 5 — Hand-Crafted XOR Network (lines 166–196)

```
W1 = np.array([[20, -20], [20, -20]])
b1 = np.array([-10, 30])
W2 = np.array([[20], [20]])
b2 = np.array([-30])
```

Hard-coded weight matrices and bias vectors — typed in directly, not learned by training. **W1** is a 2×2 array (2 inputs feeding into 2 hidden units); **W2** is 2×1 (2 hidden units feeding into 1 output).

MATH

$h = \text{sigmoid}(x \times W_1 + b_1)$
 $\text{output} = \text{sigmoid}(h \times W_2 + b_2)$
(*x is length 2, W_1 is 2×2, h is length 2, W_2 is 2×1, output is a single number*)

```
def forward(x):
    h = sigmoid(x @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    return out[0]
```

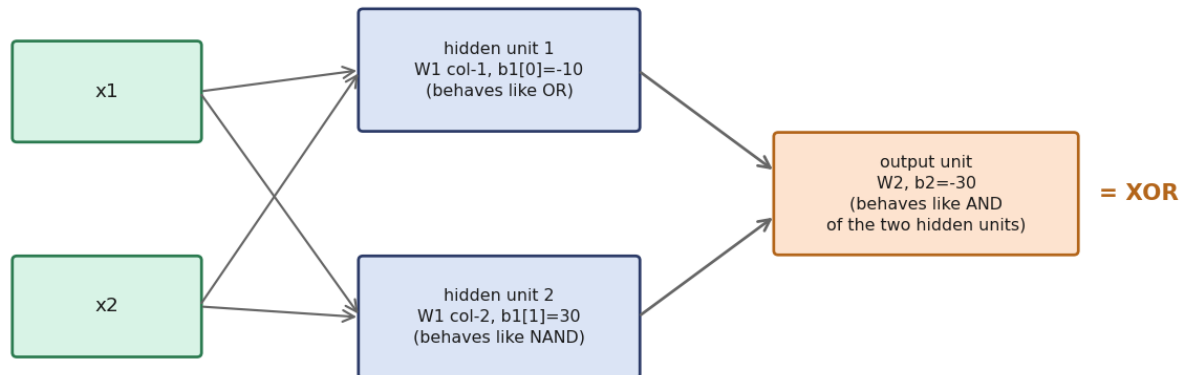
This **forward** is a **local (nested) function**, defined inside **demo_handcrafted_xor_network()** — it exists only within that function's scope, and it can see **W1**, **b1**, **W2**, **b2** from the enclosing scope automatically (a **closure**) without them being passed in as arguments. The **@** operator is **matrix multiplication** — distinct from *****, which multiplies elementwise. **x @ W1** takes a shape-(2,) input and a shape-(2,2) matrix and produces a shape-(2,) result — the weighted sum flowing into each hidden unit, computed all at once instead of via a manual loop. Adding **b1** broadcasts the bias onto that result; **sigmoid(...)** then squashes it elementwise. **out[0]** extracts the plain scalar from what would otherwise remain a 1-element array (because **b2** has shape (1,), the whole computation stays array-shaped until this final index).

```
for xi, yi in zip(X_xor, y_xor):
    pred = forward(xi)
    pred_class = 1 if pred >= 0.5 else 0
    correct += (pred_class == yi)
```

pred_class = 1 if pred >= 0.5 else 0 thresholds the sigmoid's continuous 0–1 output into a hard class label at the standard 0.5 cutoff, using the same conditional-expression syntax seen in the **Perceptron.forward()** method. **correct += (pred_class == yi)** again relies on **True/False** acting as **1/0** under addition to accumulate a running correct-count.

SYSTEM DIAGRAM

Hand-crafted XOR network: architecture (Segment 5)



Every arrow is one number in $W1/b1/W2/b2$ -- typed in by hand, not learned.
code: $h = \text{sigmoid}(x @ W1 + b1)$ then $out = \text{sigmoid}(h @ W2 + b2)$

The 4 hard-coded arrays (**$W1$** , **$b1$** , **$W2$** , **$b2$**) as an actual network architecture: 2 inputs cross into 2 hidden units (one behaving like OR, one like NAND — verify this yourself with the truth table), and the output unit combines them like AND. That specific combination is mathematically identical to XOR — the whole reason this hand-picked shape can solve a problem no single perceptron can.

Segment 6 — TinyNet Class: Manual Backprop (lines 203–254)

```
def __init__(self, n_in, n_hidden, lr=0.5):
    self.W1 = np.random.randn(n_in, n_hidden) * 0.5
    self.b1 = np.zeros(n_hidden)
    self.W2 = np.random.randn(n_hidden, 1) * 0.5
    self.b2 = np.zeros(1)
    self.lr = lr
```

np.random.randn(n_in, n_hidden) draws random numbers from a standard normal distribution (mean 0, standard deviation 1) into a shape-(n_in, n_hidden) array — this is the random weight initialization. *** 0.5** scales those values down so the initial weights start small. **np.zeros(n_hidden)** initializes the biases at exactly 0, a common convention — unlike weights, biases don't need randomness to break symmetry between hidden units.

MATH

$$\begin{aligned}z_1 &= X \times W_1 + b_1 \\a_1 &= \text{sigmoid}(z_1) \\z_2 &= a_1 \times W_2 + b_2 \\a_2 &= \text{sigmoid}(z_2) = y_{\text{pred}}\end{aligned}$$

```
def forward(self, X):
    self.z1 = X @ self.W1 + self.b1
    self.a1 = sigmoid(self.z1)
    self.z2 = self.a1 @ self.W2 + self.b2
    self.a2 = sigmoid(self.z2)
    return self.a2
```

Same @ matrix-multiply pattern as Segment 5, but now **X** is the **entire dataset at once** (shape (300, 2) for the moons data later on), so **X @ self.W1** computes the hidden layer's weighted sums for all 300 examples simultaneously — no Python loop over individual examples anywhere in this method. Storing results as **self.z1**, **self.a1**, etc. (rather than plain local variables) matters: **backward()** needs these exact cached intermediate values to compute gradients, since redoing the forward pass inside **backward()** would be wasteful and error-prone.

MATH

Binary cross-entropy loss: $L = -(1/m) \times \text{sum}[y \times \log(a_2) + (1-y) \times \log(1-a_2)]$
Its derivative w.r.t. z_2 , combined with the sigmoid derivative, simplifies beautifully to just:
$$dz_2 = a_2 - y$$

```
def backward(self, X, y):
    m = X.shape[0]
    y = y.reshape(-1, 1)
    dz2 = self.a2 - y
```

X.shape returns a tuple of an array's dimensions, e.g. **(300, 2)**; **[0]** grabs the first number — 300, the number of training examples — used later to average gradients across the batch. **y.reshape(-1, 1)** reshapes a 1D array of shape (300,) into a 2D column of shape (300, 1); the **-1** tells NumPy to auto-infer that dimension from the total element count. This matters because **self.a2** (the prediction) is shape (300, 1) — subtracting a (300,) array from a (300, 1) array without reshaping would silently broadcast into the wrong shape instead of raising an error.

MATH

$$\begin{aligned}dW_2 &= (1/m) \times a_1^T \times dz_2 \\db_2 &= \text{mean}(dz_2) \\da_1 &= dz_2 \times W_2^T \text{ (chain rule: how loss flows back into the hidden layer)} \\dz_1 &= da_1 \times a_1 \times (1 - a_1) \text{ (sigmoid'(z) = } a(1-a)\text{)} \\dW_1 &= (1/m) \times X^T \times dz_1 \\db_1 &= \text{mean}(dz_1)\end{aligned}$$

```
dw2 = self.a1.T @ dz2 / m
db2 = dz2.mean(axis=0)
da1 = dz2 @ self.W2.T
dz1 = da1 * self.a1 * (1 - self.a1)
dw1 = X.T @ dz1 / m
db1 = dz1.mean(axis=0)
```

.T transposes a matrix — swaps its rows and columns — needed here so the dimensions line up correctly for **@** to produce a gradient array the same shape as the weight matrix it corresponds to. **dz2.mean(axis=0)** averages down **axis=0** — across all 300 examples, i.e. down the rows — collapsing a per-example gradient into a single gradient value per bias term. Every line here is standard chain-rule calculus expressed directly as NumPy matrix operations, no explicit loop over the 300 examples anywhere.

MATH

$$\begin{aligned}W_1 &= W_1 - lr \times dW_1 & b_1 &= b_1 - lr \times db_1 \\W_2 &= W_2 - lr \times dW_2 & b_2 &= b_2 - lr \times db_2\end{aligned}$$

(standard gradient descent: $\text{parameter} = \text{parameter} - lr \times \text{gradient}$)

```
self.W1 -= self.lr * dw1
self.b1 -= self.lr * db1
self.W2 -= self.lr * dw2
self.b2 -= self.lr * db2
```

The gradient-descent parameter update. **-=** subtracts because the goal is to move **against** the gradient direction (downhill on the loss surface) to reduce the loss, and the step size is scaled by **self.lr**, the learning rate.

```
def train(self, X, y, epochs=3000):
    losses = []
    eps = 1e-9
    for _ in range(epochs):
        pred = self.forward(X)
        y_col = y.reshape(-1, 1)
```

eps = 1e-9 is a tiny constant — **1e-9** is scientific notation for 0.000000001 — added inside upcoming **log()** calls purely to prevent **log(0)**, which is negative infinity and would corrupt the loss if any prediction ever landed exactly on 0 or 1. **y_col = y.reshape(-1, 1)** applies the same (m,) → (m, 1) reshape used inside **backward()**, this time so the loss formula below lines up correctly against **pred**.

MATH

$$L = -(1/m) \times \text{sum}[y \times \log(y_pred + eps) + (1-y) \times \log(1-y_pred + eps)]$$

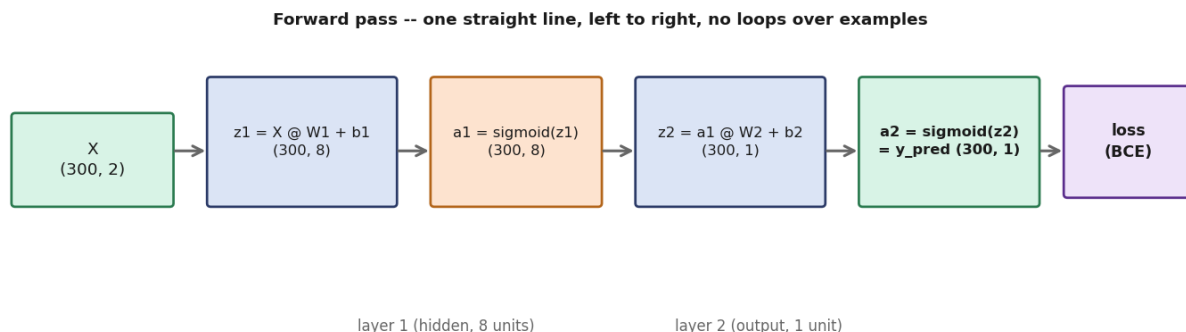
(eps is added purely so log(0) never happens; it does not change the result meaningfully)

```
        loss = -np.mean(y_col * np.log(pred + eps) + (1 - y_col) * np.log(1 - pred + eps))
        losses.append(loss)
        self.backward(X, y)
    return losses
```

This single line is the binary cross-entropy loss formula written directly in NumPy: **np.mean(...)** averages the per-example loss across all 300 examples, and the leading - negates the result (cross-entropy is conventionally reported as a positive number, but the raw log-likelihood inside is negative). **losses.append(loss)** runs **before** **self.backward(X, y)** deliberately — recording each epoch's loss *before* that epoch's weight update means **losses[0]** reflects the network's performance before any training happened at all.

SYSTEM DIAGRAM

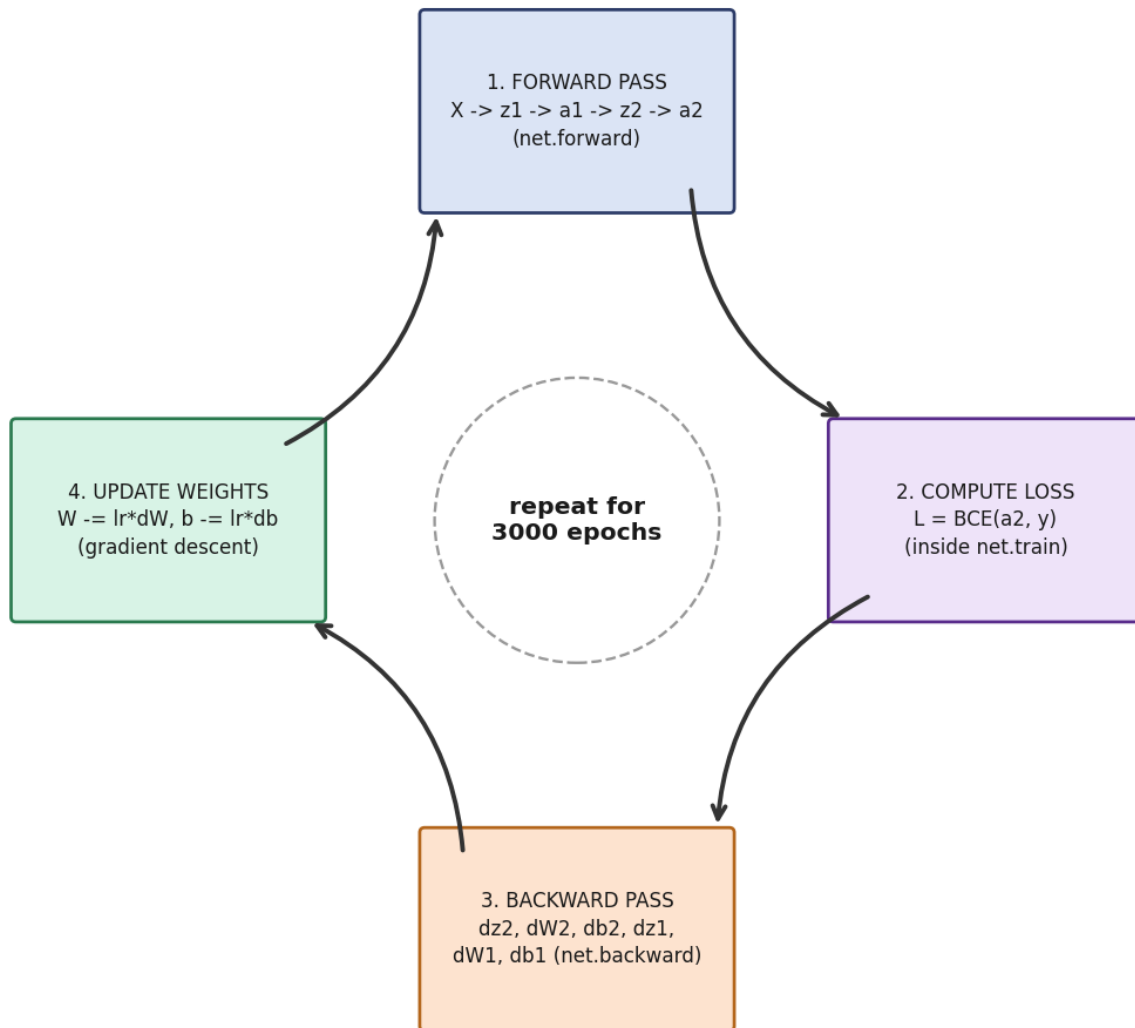
TinyNet forward pass: system diagram (Segment 6)



forward() as a system: **X** flows left to right through exactly the 4 math lines from this segment — **z1**, **a1**, **z2**, **a2** — every arrow a single matrix operation applied to all 300 examples simultaneously, ending in the loss box that **train()** computes next.

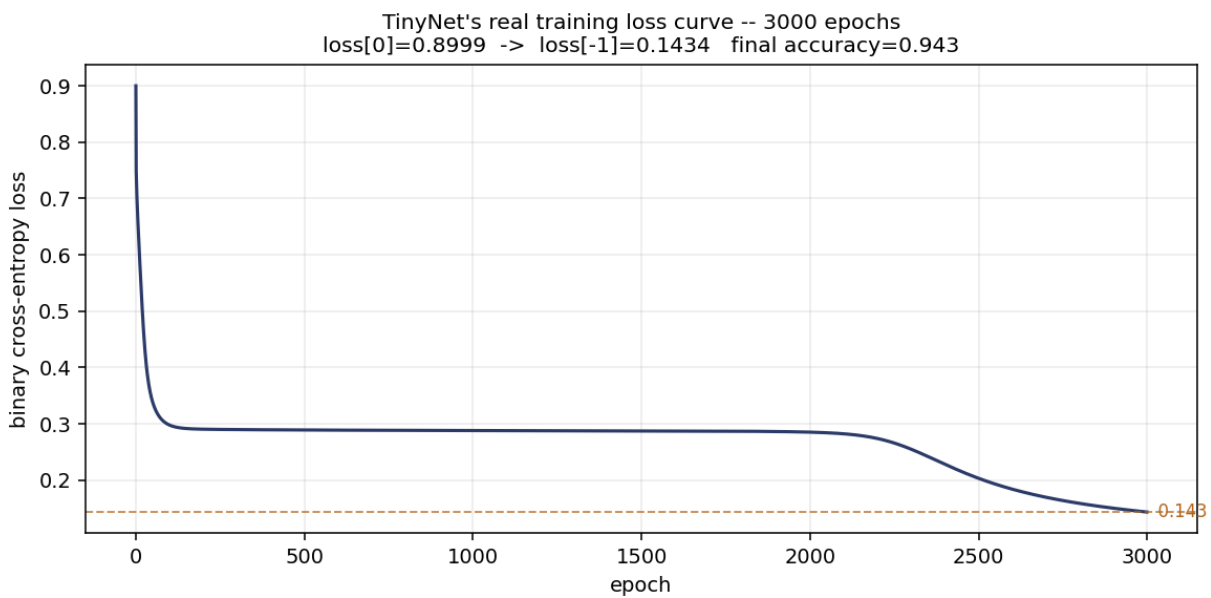
SYSTEM DIAGRAM

The training work cycle (Segment 6, TinyNet.train)



The **work cycle** that **train()**'s epoch loop actually runs, matching the 4 code blocks and math boxes just covered: forward pass produces **a2**, the loss box scores it, **backward()** turns that score into 6 gradients by the chain rule, and the update step nudges every weight downhill — then the whole cycle repeats, 3000 times, for this script's real run.

SYSTEM DIAGRAM



Not a mockup — this is **this exact script**, rerun fresh, plotting **losses** (the list **train()** builds one **loss.append(loss)** at a time) against epoch number. The flat plateau around epoch 200–2200 is real: this is what it visually looks like when gradient descent is still working but has hit a slow region of the loss surface, before finding a faster path down.

Segment 7 — demo_moons_comparison() (lines 257–305)

```
X_moons, y_moons = make_moons(n_samples=300, noise=0.2, random_state=42)
```

make_moons(...) returns two values at once, unpacked directly into **X_moons** (the 300×2 coordinate array) and **y_moons** (the 300 class labels). **noise=0.2** adds random jitter so the two crescent shapes aren't perfectly clean. **random_state=42** seeds this specific function's own randomness independently of the earlier global **np.random.seed(42)** — scikit-learn functions that accept **random_state** don't rely on NumPy's global seed.

MATH

$$\text{accuracy} = (1/m) \times \text{sum over } i \text{ of } [y_{\text{pred}_i} == y_i]$$

```
net_pred = (net.forward(X_moons) >= 0.5).astype(int).flatten()
net_acc = (net_pred == y_moons).mean()
```

net.forward(X_moons) >= 0.5 — comparing an array against a scalar produces a **boolean array** (elementwise True/False), one entry per prediction, shape (300, 1). **.astype(int)** converts True/False into 1/0. **.flatten()** collapses the (300, 1) shape down to plain (300,), so it can be compared directly against **y_moons** (also shape (300,)). **(net_pred == y_moons).mean()** — elementwise equality produces another boolean array; **.mean()** on booleans treats True as 1.0 and False as 0.0, so the mean is literally the fraction of correct predictions — i.e. accuracy — all computed in one line, no explicit counting loop.

```
xx, yy = np.meshgrid(
    np.linspace(X_moons[:, 0].min() - 0.5, X_moons[:, 0].max() + 0.5, 200),
    np.linspace(X_moons[:, 1].min() - 0.5, X_moons[:, 1].max() + 0.5, 200),
)
```

X_moons[:, 0] uses NumPy 2D indexing: the **:** means "all rows," and **0** means "column 0" — so this pulls out just the x-coordinates of every point (**X_moons[:, 1]** similarly gives y-coordinates). **.min() - 0.5 / .max() + 0.5** finds the data's range and pads it by 0.5 on each side, so the plotted background extends a bit beyond the actual data points. **np.meshgrid(a, b)** takes two 1D arrays and produces two 2D grids (**xx, yy**) representing every combination of x and y — the standard way to build a background grid for a decision-boundary plot.

```
grid = np.c_[xx.ravel(), yy.ravel()]
```

.ravel() flattens a 2D array back down into 1D. **np.c_[a, b]** stacks two 1D arrays together as columns into a 2D array — so **grid** ends up shaped (40000, 2): every (x, y) point on the grid, in the same (n_points, 2) shape the network's **forward()** expects as input.

```
net_zz = net.forward(grid).reshape(xx.shape)
```

Runs the trained network on all 40,000 grid points in a single call (fully vectorized, no loop), then **.reshape(xx.shape)** folds the flat result back into the 2D grid shape so it can be plotted as a filled contour.

```
perc_zz = np.array([perc.forward(pt) for pt in grid]).reshape(xx.shape)
```

The **Perceptron.forward()** method only accepts one row at a time — it has no vectorized batch support — so this line falls back to a **list comprehension** that loops over every point in **grid** individually, wraps the resulting Python list back into a NumPy array with **np.array(...)**, then reshapes it the same way as the network's output above.

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
panels = [
    (axes[0], perc_zz, "Single perceptron (linear)", perc_acc),
    (axes[1], net_zz, "2-layer network (non-linear)", net_acc),
]
```

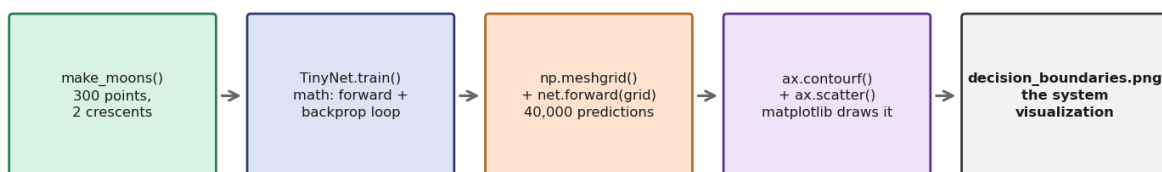
panels is a list of 4-element tuples, bundling together everything one subplot needs to be drawn — its axis object, its background grid values, its title, its accuracy — so the loop that follows can draw both panels identically instead of writing the drawing code out twice.

```
for ax, zz, title, acc in panels:
    colors_ = ["#F5C4B3", "#9FE1CB"]
    ax.contourf(xx, yy, zz, levels=[-1, 0.5, 2],
                colors=colors_, alpha=0.6)
    ax.scatter(X_moons[:, 0], X_moons[:, 1], c=y_moons, cmap="coolwarm",
                s=12, edgecolors="k", linewidths=0.3)
    ax.set_title(f"{title}\naccuracy={acc:.3f}")
```

for ax, zz, title, acc in panels: unpacks each 4-tuple into four loop variables per iteration. **ax.contourf(xx, yy, zz, levels=[-1, 0.5, 2], colors=[...])** draws filled regions: wherever **zz** falls between -1 and 0.5 it colors that area the first color, between 0.5 and 2 the second — this is exactly what paints the pink/green decision-boundary background. **ax.scatter(..., c=y_moons, cmap="coolwarm", s=12, edgecolors="k", linewidths=0.3)** plots each data point: **c=y_moons** colors points by class label using the "coolwarm" color map, **s=12** sets marker size, and **edgecolors/linewidths** add a thin black outline per dot for visibility against the colored background. Finally, **\n** inside the f-string is a newline character, wrapping the title onto two lines, and **{acc:.3f}** formats the float to exactly 3 decimal places.

SYSTEM DIAGRAM

Segment 7 pipeline: how code + math becomes the final chart



Zooming out: this is the full pipeline this segment's code runs, start to finish — from a synthetic dataset, through the math-and-code loop covered in Segment 6, through a 40,000-point grid evaluation, to the matplotlib calls that actually paint pixels. Every box here is code you have now read line by line; **decision_boundaries.png** is what all of it together produces.

Segment 8 — main() and the Entry-Point Guard (lines 329–345)

```
def main():
    demo_perceptron_and_gate()
    demo_perceptron_xor_failure()
    demo_activation_functions()
    demo_handcrafted_xor_network()
    demo_moons_comparison()
    note_on_pytorch()
```

A single function that calls every demo in order — this is the one place in the file that defines "what running this script actually does," top to bottom.

```
if __name__ == "__main__":
    main()
```

A standard Python idiom. `__name__` is a special variable Python sets automatically for every module: it equals `"__main__"` only when the file is executed directly (e.g. `python day36_neural_network_fundamentals.py`), but equals the module's own name if the file is instead **imported** by another script (the way Day 34 imports `add_family_size` from Day 33). This guard means **main()** — and therefore every demo, every print statement, every saved PNG — only fires on direct execution; importing this file elsewhere would not silently re-run all six demos.

Quick Syntax Glossary — Constructs That Repeat Throughout

`@` — Matrix multiplication between two arrays (not elementwise — use `*` for elementwise multiplication).

`.T` — Transpose: swaps the rows and columns of a 2D array.

`array[:, i]` — Select column `i`, every row — 2D NumPy indexing syntax.

`.reshape(-1, 1)` — Reshape into a single column vector; `-1` tells NumPy to auto-infer that dimension from the total element count.

`.mean(axis=0)` — Average down the rows (a per-column average). Omitting **axis** averages every element into one single number.

`f"{x:.3f}"` — Format spec inside an f-string: render the float `x` to exactly 3 decimal places.

`f"{x:8s}"` — Format spec: left-pad the string `x` to a width of 8 characters, for column alignment in printed output.

`zip(a, b)` — Iterate over two sequences in parallel, pairing up elements by matching position.

`[expr for item in iterable]` — List comprehension — builds a brand-new list by evaluating **expr** once per item.

`(expr for item in iterable)` — Generator expression — same idea as a list comprehension, but produces values lazily, one at a time, instead of building the full list in memory up front.

`True == 1, False == 0` — Booleans are integers under the hood in Python. This is why **.mean()** on a boolean array gives a fraction (accuracy), and **sum()** over a sequence of comparisons counts how many were True.

End of Day 36 syntax reference. For the neural-network concepts these lines implement (perceptrons, activation functions, backpropagation, why XOR needs depth), see the main Day 36 study guide PDF.